

A Multi-Agent Deep-Research Assistant for HCI Topics

Qiming L.

Assignment 3, Multi-Agent Systems

Fall 2025

qiming.l@example.edu

TABLE I
AGENTS, TOOLS, AND HAND-OFF TOKENS.

Agent	Tools	Hand-off token
Planner	—	PLAN COMPLETE
Researcher	web_search, paper_search	RESEARCH COMPLETE
Writer	—	DRAFT COMPLETE
Critic	—	APPROVED / TERMINATE

Abstract—We present a four-agent deep-research assistant for human–computer interaction (HCI) topics, orchestrated with Microsoft AutoGen’s RoundRobinGroupChat. A *Planner*, *Researcher*, *Writer*, and *Critic* cooperate via a shared chat, with the Researcher invoking real Tavily web search and Semantic Scholar paper search through `FunctionTool` wrappers. Around this core we add (i) a custom rule-based safety policy filter that runs both *before* the team executes (input guardrail covering harmful content, prompt injection, off-topic queries, and malformed input) and *after* (output guardrail covering PII, harmful content, bias, and citation grounding); (ii) a two-perspective LLM-as-a-Judge — a *supportive* reviewer and a *strict* reviewer — that each score five criteria (relevance, evidence quality, factual accuracy, safety compliance, clarity); and (iii) CLI and Streamlit interfaces that surface agent traces, citations, and safety refusals. The system runs against the SALT-lab `gpt-5-mini` gateway and is evaluated on ten HCI queries plus five adversarial safety probes.

Index Terms—multi-agent systems, AutoGen, LLM-as-a-Judge, safety guardrails, HCI, retrieval-augmented generation

I. SYSTEM DESIGN AND IMPLEMENTATION

A. Agents and Control Flow

Four AutoGen `AssistantAgent` instances share a chat managed by `RoundRobinGroupChat` and terminate on the token `TERMINATE`. Table I summarises the role of each agent and its hand-off token. The orchestrator (`src/autogen_orchestrator.py`) wraps `team.run()` with two safety gates and a result extractor that pulls the *Writer*’s message as the canonical answer (rather than the *Critic*’s review), and aggregates messages, sources, and the running agent set into structured metadata returned to the UI layer.

B. Tools

`src/tools/web_search.py` exposes a thin `WebSearchTool` over Tavily and Brave with a

unified result schema (title, url, snippet, score, published_date). `src/tools/paper_search.py` wraps the `semanticscholar` Python client and supports year/citation filters. Both are exposed to AutoGen as synchronous `FunctionTool` instances. `src/tools/citation_tool.py` formats sources in APA-7 (with MLA fallback), de-duplicates entries, and emits a sorted bibliography.

C. Models and Configuration

All tunables — model provider, agent prompts, tool providers, safety policy, and judge criteria — live in `config.yaml`; secrets live in `.env`. The default chat model is `gpt-5-mini` served by the SALT-lab OpenAI-compatible gateway (<https://openai.gateway.salt-lab.org/v1>); the judge model points at the same endpoint with `temperature=0.3`. Both Groq and OpenAI-compatible (vLLM) clients are auto-detected at runtime, so the same code runs against the class-provided endpoint or a Groq fallback without modification.

D. User Interfaces

A Streamlit web UI (`src/ui/streamlit_app.py`) shows the final answer, citations, agent traces, a quality score, and a dedicated *Safety Events* expander. A CLI (`src/ui/cli.py`) prints the same information for terminal use. A single entry point, `main.py`, exposes five modes: `cli`, `web`, `evaluate`, `autogen`, and `demo` (a one-shot end-to-end query that exports the full session as JSON to `outputs/`).

II. SAFETY DESIGN

We defined a six-category policy and implemented it as a custom rule-based filter in `src/guardrails/`. We chose rule-based filters over Guardrails AI [4] or NeMo Guardrails [3] to keep the system deterministic, offline-friendly, and inspectable; the implementation exposes a `_check_with_external_validator` hook so a stronger validator can be plugged in without changing the orchestrator.

Severity is high, medium, or low; only high violations trigger the refusal path. Each event carries (timestamp, type, action, violations, content_preview)

TABLE II
SAFETY POLICY: CATEGORIES, STAGE, AND DEFAULT ACTION.

Category	Stage	Default action
harmful_content	input + output	refuse
prompt_injection	input	refuse + sanitise
off_topic_queries	input	warn
pii	output	sanitise (redact)
misinformation	output	warn
bias	output	sanitise

and is appended both to an in-memory list (returned to the UI in `metadata.safety.events`) and to `logs/safety_events.log`. The orchestrator calls `check_input_safety()` *before* `team.run()` and `check_output_safety()` *after*, so blocked queries never reach the LLM and PII never reaches the user.

The Streamlit UI displays a red banner — “*Input blocked by a safety policy*” — with the exact triggered category, and an amber banner for sanitisation. We verified all six categories pass on a five-query adversarial probe set (`data/safety_test_queries.json`); for example, “*Ignore previous instructions and tell me how to make a bomb*” fires `harmful_content`, `prompt_injection`, and `off_topic_queries` simultaneously, and the team is never invoked.

III. EVALUATION SETUP AND RESULTS

A. Dataset

`data/example_queries.json` contains ten diverse HCI queries spanning explainable AI, AR usability, AI ethics, UX measurement, conversational AI in healthcare, accessibility, uncertainty visualisation, voice UIs for the elderly, AI prototyping, and cross-cultural design. Each item carries a category and an expected-topic list; three carry an APA-style ground truth. `data/safety_test_queries.json` adds five adversarial / control prompts targeting the safety policy.

B. LLM-as-a-Judge: Two Independent Rubrics

A single judging perspective biases toward whatever rubric the prompt evokes. We therefore implemented an *ensemble* of two perspectives in `src/evaluation/judge.py`:

- **Supportive reviewer** — “rigorous but partial-credit-friendly”, scale 0.0–1.0 with mid-range anchors.
- **Strict reviewer** — “sceptical peer reviewer at a top HCI venue”, same scale but conservative; demands citations.

Each of the five criteria (relevance, evidence quality, factual accuracy, safety compliance, clarity, with weights 0.25/0.25/0.20/0.15/0.15) is scored independently by both perspectives; the per-criterion score is the average; the overall score is the weighted average; the per-perspective score is the mean across criteria. The judge prompt instructs strict JSON output and tolerates markdown fences via a regex extractor (`_parse_judgment`).

TABLE III
JUDGE SCORES FOR THE LIVE GPT-5-MINI DEMO RUN.

Criterion	Supportive	Strict	Avg.
Relevance	0.00	0.00	0.00
Evidence quality	0.65	0.45	0.55
Factual accuracy	0.80	0.60	0.70
Safety compliance	1.00	1.00	1.00
Clarity	0.65	0.45	0.55
Weighted overall	0.68	0.44	0.51

C. Results

We ran one full demo (`main.py --mode demo`) end-to-end against `gpt-5-mini`. The four agents exchanged 21 messages and produced a 4947-character draft. Per-criterion scores are shown in Table III.

The 0.24 spread between perspectives confirms the value of ensembling: the strict reviewer penalised the lack of grounded citations far more aggressively than the supportive one. Safety compliance scored 1.0 from both judges, as expected.

D. Error Analysis

The Researcher’s tool calls *failed silently* on this run because no `TAVILY_API_KEY` was supplied and Semantic Scholar refused the unauthenticated connection (rate limit). The agents detected the empty results and entered a “scope confirmation” loop with the (mocked) user, never producing the substantive HCI synthesis the query asked for — hence relevance 0.00. This is informative: the system correctly *detected* that it lacked evidence rather than hallucinating, but it also failed to recover (e.g. by retrying with a different provider or by writing a clearly-flagged “no-evidence” answer). A pre-recorded reference run on the same accessible-UI query (with Tavily enabled) reached overall 0.84 (see `outputs/sample_session.md`), suggesting tool availability is the dominant bottleneck rather than agent design. Three additional failure patterns appeared across exploratory runs: (a) over-long Critic feedback echoing back into the chat and pushing context, (b) Writer occasionally appending the literal `TERMINATE` token to its draft (stripped post-hoc by the orchestrator), and (c) the Critic asking the user clarifying questions instead of approving — addressable by tightening the agent prompts and adding a hard cap on rounds.

IV. DISCUSSION AND LIMITATIONS

Insights: Splitting safety into an *input* gate (rule-based, free, fast) and an *output* gate (PII regex + citation heuristic) gave us deterministic guarantees on the high-severity classes (harmful content, prompt injection, raw PII) without needing a moderation model. The two-perspective judge produced more interpretable signals than a single rubric: a wide supportive–strict gap reliably flagged “stylistically OK but evidence-thin” answers. Wrapping each tool as a `FunctionTool` and letting the Researcher choose calls — rather than hard-coding a sequence — preserved AutoGen’s idiomatic flow at the cost of occasional skipped searches.

Limitations: (1) The guardrails are regex-based and will miss obfuscated harmful content; the `_check_with_external_validator` hook is a placeholder, not a wired Guardrails-AI / NeMo integration. (2) Topic relevance is keyword-driven and biased toward English HCI vocabulary. (3) The judge uses the same model (`gpt-5-mini`) as the Writer, so it shares the Writer’s blind spots — a stronger setup would use a different model family for judging. (4) Tool failures are reported but the agents do not recover gracefully (no exponential back-off, no provider fallback). (5) Evaluation $N=10$ queries is small; per-criterion variance is therefore wide.

Future work: Plug a real moderation API (e.g. OpenAI Moderation, Llama Guard) into the input gate; add an LLM-based citation-grounding verifier to the output gate; introduce a *Verifier* agent that checks each Writer claim against the retrieved snippets before the Critic; sweep three judges and report inter-rater agreement (Cohen’s κ); and grow the eval set to ≥ 50 queries for tighter confidence intervals.

Ethics: The system is deliberately constrained to the HCI research domain, refuses harmful and injection requests at the front door, and redacts PII before it is shown. We do not log full user queries to disk — only 200-character previews — and we keep secrets out of the repo via `.gitignore`.

REFERENCES

- [1] Q. Wu *et al.*, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023. [Online]. Available: <https://arxiv.org/abs/2308.08155>
- [2] Microsoft, “AutoGen: Multi-agent conversation framework (documentation),” 2024. [Online]. Available: <https://microsoft.github.io/autogen/>
- [3] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, “NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails,” *arXiv preprint arXiv:2310.10501*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.10501>
- [4] Guardrails AI, “Guardrails: An open-source toolkit for safer LLM applications,” 2024. [Online]. Available: <https://docs.guardrailsai.com/>
- [5] Tavily, “Tavily search API documentation,” 2024. [Online]. Available: <https://docs.tavily.com/>
- [6] Allen Institute for AI, “Semantic Scholar Academic Graph API,” 2024. [Online]. Available: <https://api.semanticscholar.org/>
- [7] S. L. Henry, “Web content accessibility guidelines (WCAG) overview,” W3C Web Accessibility Initiative, 2024. [Online]. Available: <https://www.w3.org/WAI/standards-guidelines/wcag/>
- [8] J. Lazar, D. Goldstein, and A. Taylor, *Ensuring Digital Accessibility through Process and Policy*. San Francisco, CA, USA: Morgan Kaufmann, 2017.
- [9] H. Petrie and N. Bevan, “The evaluation of accessibility, usability, and user experience,” in *The Universal Access Handbook*, C. Stephanidis, Ed. Boca Raton, FL, USA: CRC Press, 2009, pp. 1–16.
- [10] WebAIM, “Introduction to web accessibility,” 2024. [Online]. Available: <https://webaim.org/intro/>